Spring Boot 3.X 개발 기본

inky4832@daum.net

1. Spring Boot 기초 및 개발환경 구성

- Spring Boot 3.X 개요 및 특징
- 개발환경 구성 (Intellij IDEA)
- Spring Boot 프로젝트 생성 및 구조
- Spring Boot 기본 어노테이션
- Auto Configuration과 Starter
- 실습

1.1 Spring Boot 3.X 개요 및 특징

Spring Boot 3.X

- 개념
  - Java 기반 어플리케이션을 쉽고 빠르게 개발할 수 있도록 돕는 프레임워크
  - 최신 Java 17+ 기반으로 동작하는 차세대 버전
  - 기존 Spring Framework의 복잡한 설정 자동화
  - 클라우드, 마이크로 서비스, 대규모 트래픽 대응 등 현대적 환경을 고려

- 특징
  - 최신 기술 스택 기반 (Java 17+, Jakarta EE 10 기반)
  - 향상된 성능 및 최적화
  - 클라우드 네이티브 친화
  - 보안 및 표준 강화
  - 개발 생산성 향상

1.1 Spring Boot 3.X 개요 및 특징

장단점

- 장점
  - 빠른 개발 속도
  - 유지보수 용이
  - 대규모 서비스 대응 가능
  - 보안 강화
  - 광범위한 커뮤니티와 생태계

- 단점
  - 최신 Java 필수 (Java 17+)
  - 마이그레이션 비용
  - 복잡한 기능 구현 시 학습 곡선

1.2 개발환경 구성 (Intellij IDEA)

학습목표

- Java 21 LTS 개요
- Gradle 개요
- Intellij IDEA 개요

1.2 개발환경 구성 (Intellij IDEA)

Java 21 LTS

- 개요
  - 2023년 09월 출시
  - LTS(Long Term Support) 로 안정적 지원
  - 기업과 개발자가 가장 많이 사용될 버전

- 특징

  장기 지원 (LTS)

  새로운 언어 기능
  - 패턴 매칭 (Pattern Matching) 강화
  - Record 클래스 개선
  - Sealed 클래스/인터페이스

  성능 개선

  보안 및 표준 강화

1.2 개발환경 구성 (Intellij IDEA)

장단점

- 장점
  - 안정성
  - 코드 작성 간결화
  - 성능 향상
  - 최신 프레임워크 호환성
  - 보안 강화

- 단점
  - 최신 JDK 필요 (JDK 17+)
  - 레거시 코드 호환성 이슈

1.2 개발환경 구성 (Intellij IDEA)

다운로드 & 설치

- Oracle JDK 21

  https://www.oracle.com/java/technologies/downloads/#jdk21-windows
  운영체제 (Windows/macOS/Linux)에 맞는 설치 파일 다운로드
  C:₩Program Files₩Java₩jdk-21 에 설치됨

Linux **macOS** **Windows**

Product/file description	File size	Download
x64 Compressed Archive	186.05 MB	https://download.oracle.com/java/21/latest/jdk-21_windows-x64_bin.zip (sha256)
x64 Installer	164.42 MB	https://download.oracle.com/java/21/latest/jdk-21_windows-x64_bin.exe (sha256)
x64 MSI Installer	163.16 MB	https://download.oracle.com/java/21/latest/jdk-21_windows-x64_bin.msi (sha256)

1.2 개발환경 구성 (Intellij IDEA)

환경 변수 설정

- Windows 기준

  제어판 > 시스템 > 고급 시스템 설정 > 환경 변수

  시스템 변수 새로 만들기

  A. 변수이름: JAVA_HOME
  B. 변수값: C:₩Program Files₩Java₩jdk-21
  C. PATH 변수 편집: %JAVA_HOME%₩bin 추가

```
C:\Users\User>set JAVA_HOME
JAVA_HOME=C:\Program Files\Java\jdk-21

C:\Users\User>set PATH
Path=C:\Program Files\Java\jdk-21\bin;C:\Program Files\Commo
C:\WINDOWS\System32\Wbem;C:\WINDOWS\System32\WindowsPowerShe
zip\;C:\Program Files\Docker\Docker\resources\bin;C:\Program
ython313\Scripts\;C:\Users\User\AppData\Local\Programs\Pytho
ncher\;C:\Users\User\AppData\Local\Microsoft\WindowsApps;;C:
```

1.2 개발환경 구성 (Intellij IDEA)

환경 변수 설정

- 설치 확인

  CMD 창에서 명령어 입력

  java -version

```
C:\Users\User>java -version
java version "21.0.8" 2025-07-15 LTS
Java(TM) SE Runtime Environment (build 21.0.8+12-LTS-250)
Java HotSpot(TM) 64-Bit Server VM (build 21.0.8+12-LTS-250,
```

1.2 개발환경 구성 (Intellij IDEA)

Gradle

- 개요
  - 프로젝트를 자동으로 빌드하고 관리해 주는 빌드 도구
  - 다양한 언어 지원 (Java, Kotlin, Android 등)
  - Ant,Maven,Gradle 순으로 발전해온 현대적 빌드 시스템
  - 대규모 프로젝트 및 최신 프레임워크에서 표준처럼 사용됨

- 특징
  - 범용성
  - 자동 의존성 관리
  - 빌드 스크립트 지원
  - 고속 빌드 (증분 빌드, 빌드 캐시, 병렬 빌드 지원)
  - 플러그인 기반 확장성

1.2 개발환경 구성 (Intellij IDEA)

장단점

- 장점
  - 빠른 빌드 속도
  - 유연한 설정
  - 자동 의존성 관리
  - 현대적 표준 도구
  - 대규모 프로젝트에 적합

- 단점
  - 러닝 커브 있음
  - 초기 빌드 속도는 느림
  - 버전별 차이

1.2 개발환경 구성 (Intellij IDEA)

Gradle 설치 방법

- 수동 설치
  - https://gradle.org/releases/
  - 최신 binary-only ZIP 파일 다운로드 설치
  - GRADLE_HOME 환경 변수 설정
  - PATH에 %GRADLE_HOME%₩bin 추가

- **Gradle Wrapper 이용 (권장)**
  - **설치 불필요**
  - **Intellij IDEA 내에서 자동 다운로드**

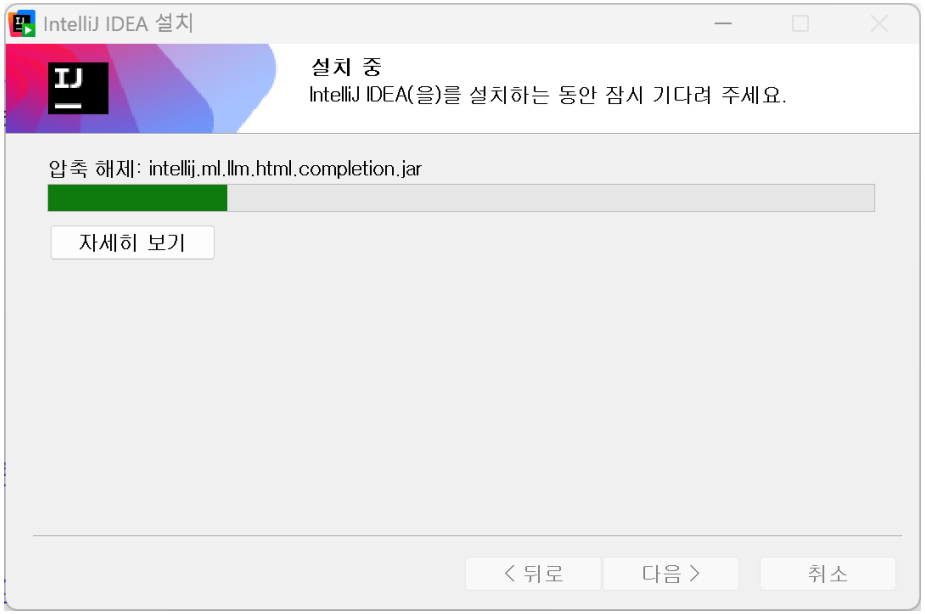
1.2 개발환경 구성 (Intellij IDEA)

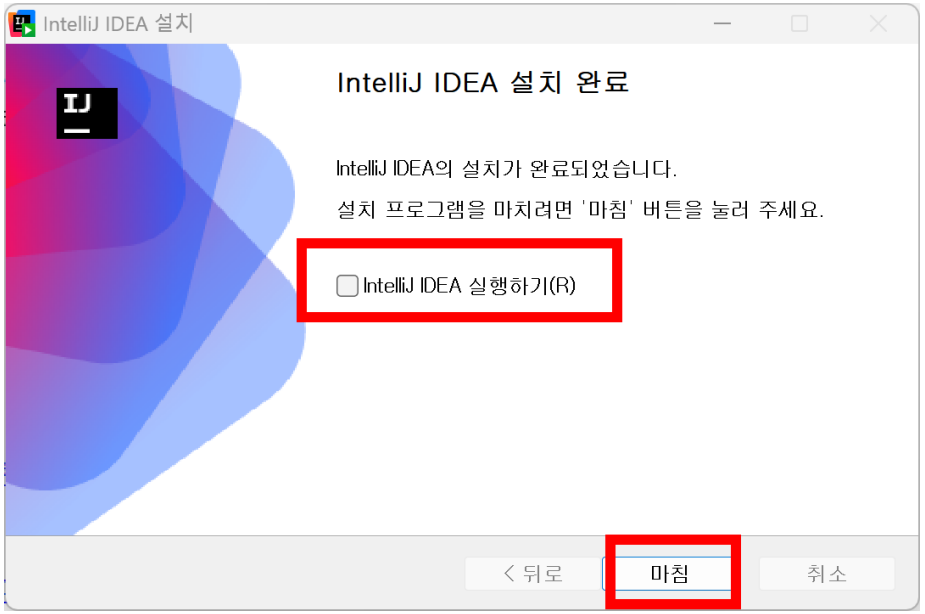
Intellij IDEA

- 공식 사이트
  https://www.jetbrains.com/idea

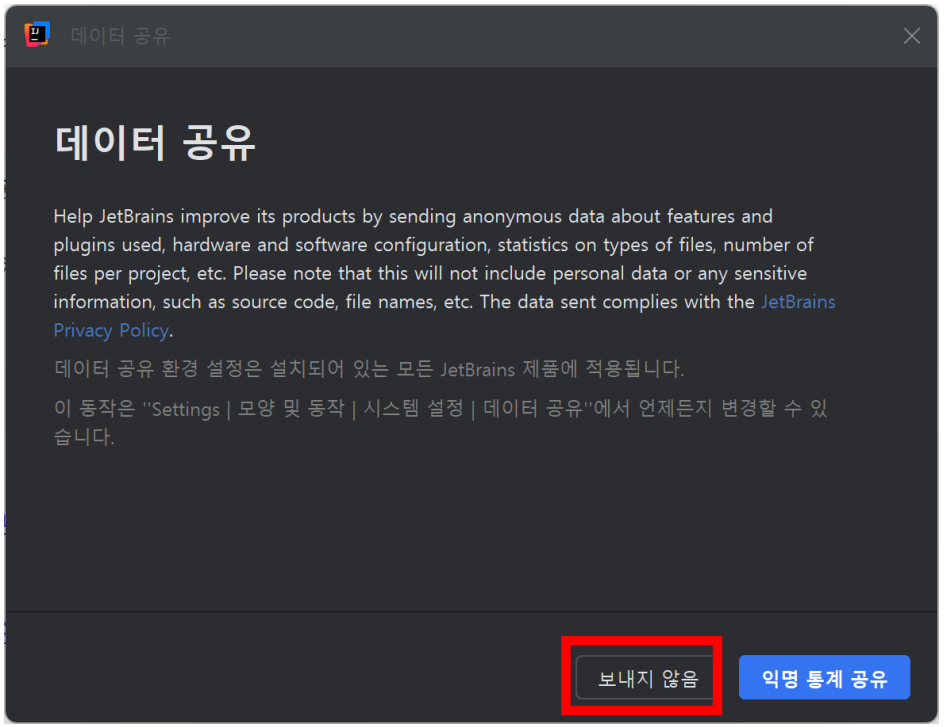


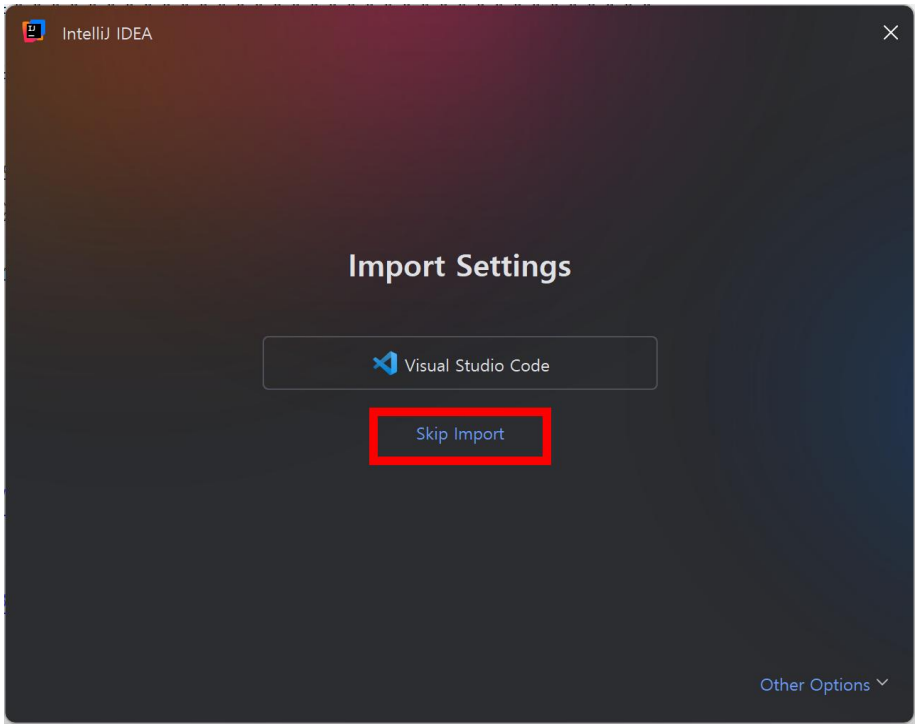
1.2 개발환경 구성 (Intellij IDEA)



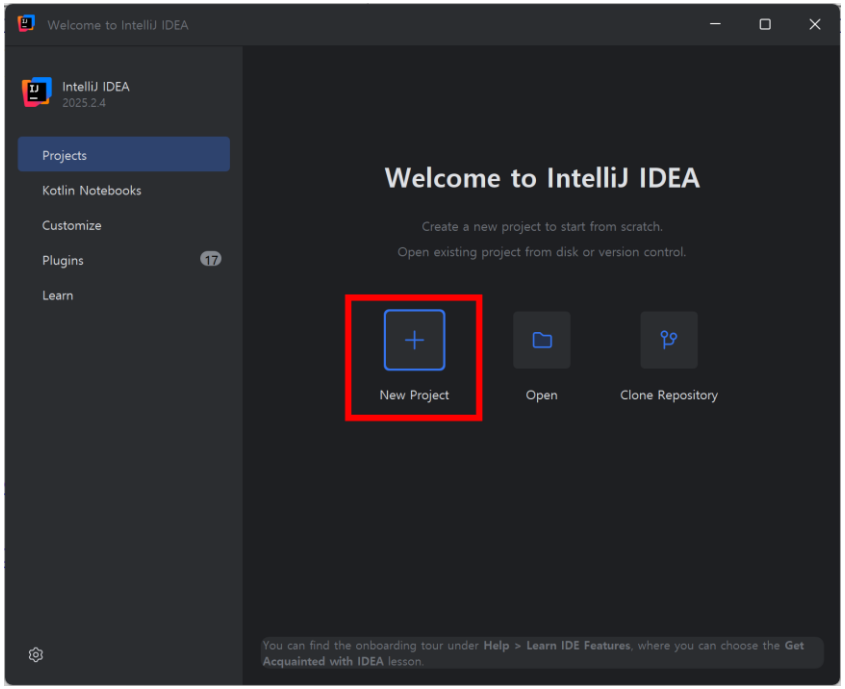


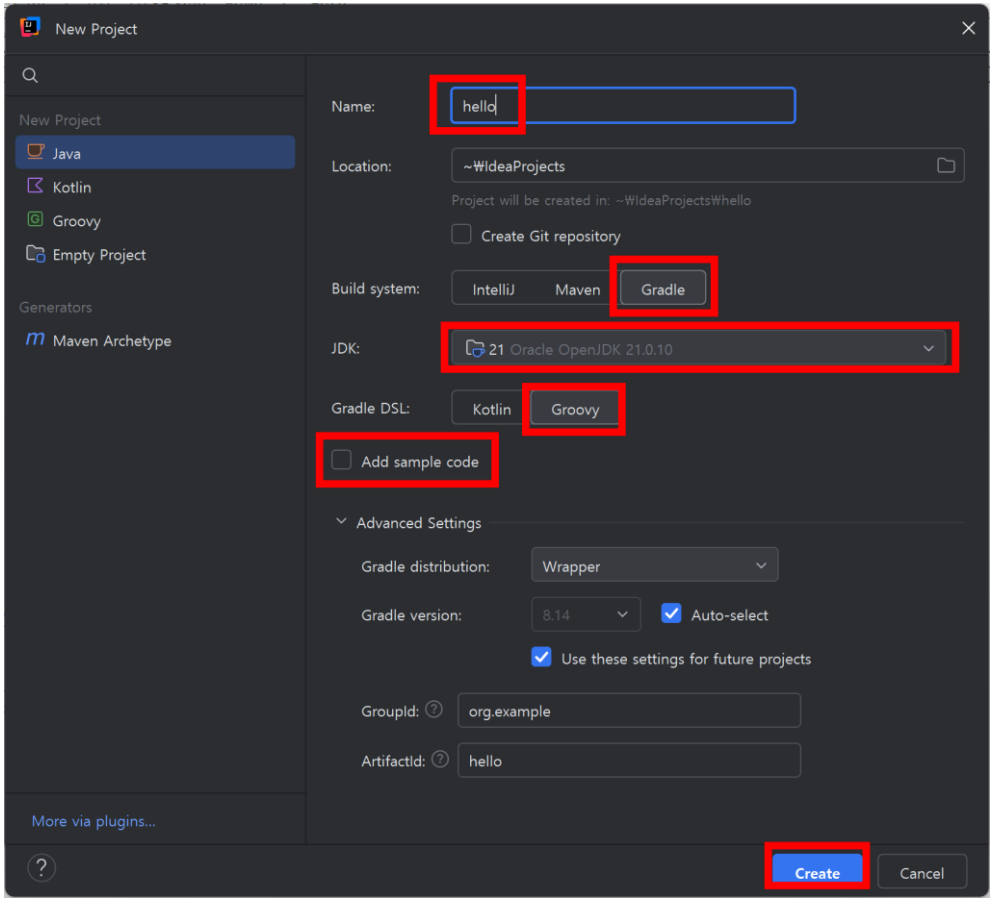
1.2 개발환경 구성 (Intellij IDEA)



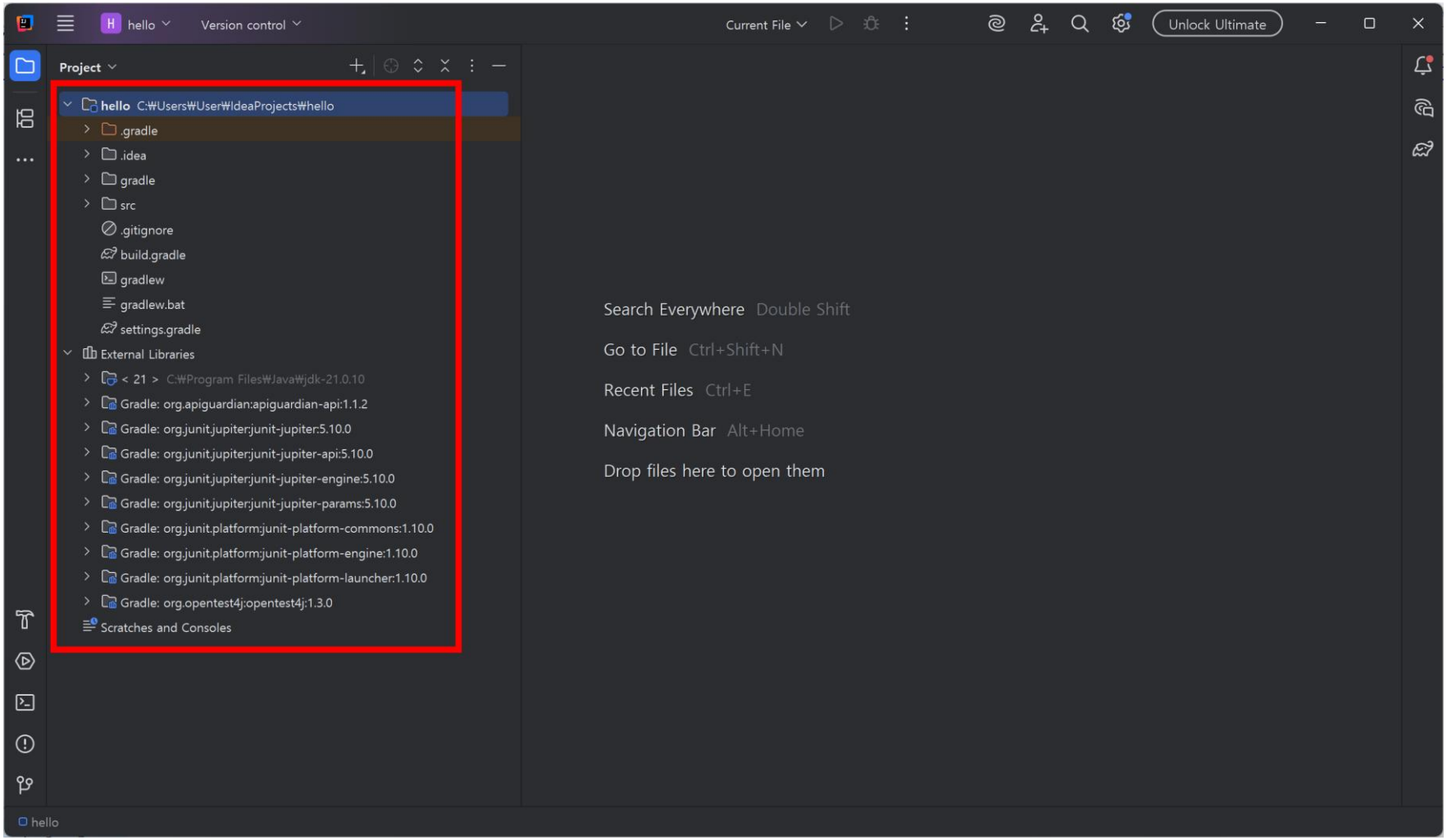


1.2 개발환경 구성 (Intellij IDEA)





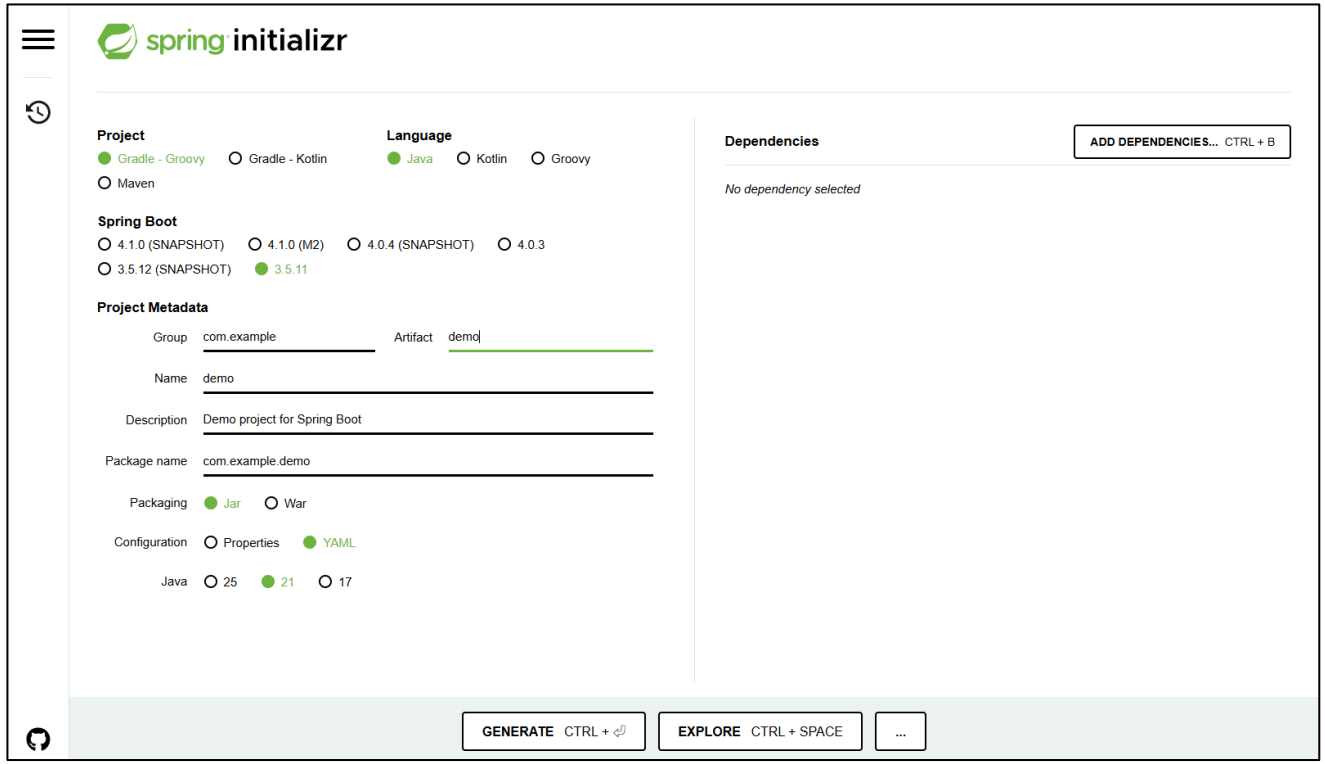
1.2 개발환경 구성 (Intellij IDEA)



1.3 Spring Boot 프로젝트 생성 및 구조

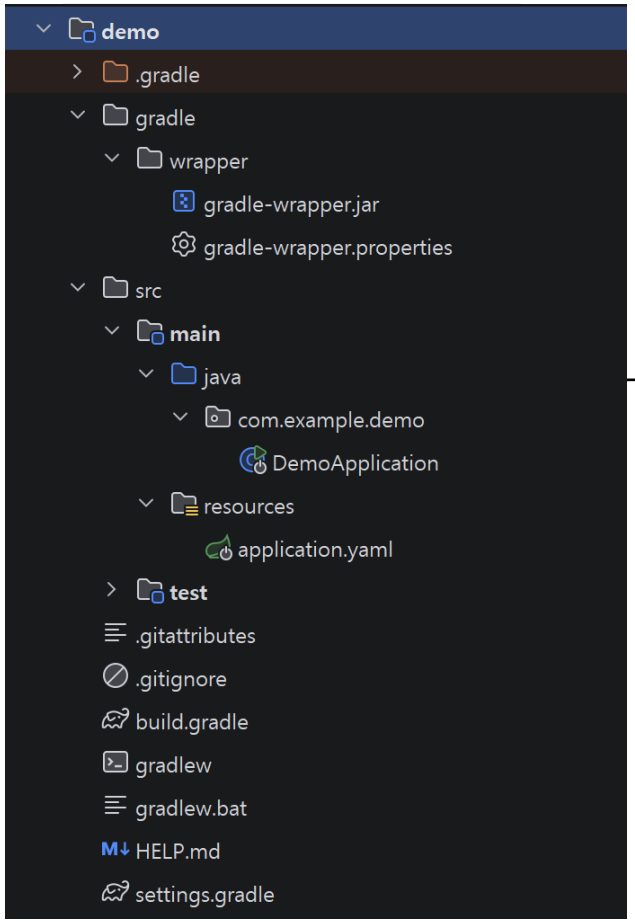
프로젝트 생성

https://start.spring.io/



1.3 Spring Boot 프로젝트 생성 및 구조

표준 프로젝트 구조



gradle-wrapper.properties : # wrapper 실행 파일

src/main/java : # 어플리케이션 기본 폴더
DemoApplication.java: # 어플리케이션 시작점 역할

src/main/resources : # 어플리케이션 공통 설정 폴더
application.yaml: # 공통 설정 파일 (.yml / .properties가능)

src/test : # 단위 테스트 폴더

build.gradle : # Gradle 스크립트 설정

gradlew : # Linux/Mac 용 Wrapper 실행 스크립트
gradlew.bat: # Windows 용 Wrapper 실행 스크립트

settings.gradle: # 루트 프로젝트/모듈 설정

1.3 Spring Boot 프로젝트 생성 및 구조

build.gradle 기본 설정 이해

```
plugins {
    id 'java'
    id 'org.springframework.boot' version '3.5.6'
    id 'io.spring.dependency-management' version '1.1.7'
}
group = 'com.example'
version = '0.0.1-SNAPSHOT'
description = 'Demo project for Spring Boot'

java {
    toolchain {
        languageVersion = JavaLanguageVersion.of(21)
    }
}
repositories {
    mavenCentral()
}
dependencies {
    implementation 'org.springframework.boot:spring-boot-starter'
    testImplementation 'org.springframework.boot:spring-boot-starter-test'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
}
tasks.named('test') {
    useJUnitPlatform()
}
```

plugins 블록
-java : 표준 Java 프로젝트용 기본 태스크/규칙 추가
ex. compileJava,test.jar 등이 활성화됨
-o.s.boot: 부트 앱에 특화된 태스크 제공
ex. bootRun, bootJar 등
-io.s.dependency-management: 의존성 및 버전 자동 관리

java Toolchain 설정
컴파일 및 테스트에 사용할 JDK 버전(21) 로 고정
로컬에 다른 JDK가 설치되어 있어도 Gradle이 JDK 21 사용

저장소 (Repositories)
의존 라이브러리 다운로드 출처 지정
mavenCentral()은 가장 보편적인 공개 저장소임

의존성(Dependencies)
implementation: 런타임/컴파일에 필요한 의존성 라이브러리
testImplementation: 테스트 코드 전용 의존성. Junit 5(Jupiter) 기반
testRuntimeOnly: 테스트 실행 시점에만 필요한 의존성

테스트 태스크 설정
JUnit Platform (JUnit 5) 기반 테스트 실행

1.3 Spring Boot 프로젝트 생성 및 구조

Gradle Wrapper 이용한 설치 & 실행

- 개념
  1. 프로젝트 실행할 때 필요한 Gradle 실행 파일이 자동 다운로드됨
  2. 권장 Gradle 설치 방식

- 특징
  - 일관성 보장
  - 설치 불필요
  - CI/CD 환경에 유리
  - 버전 관리 용이

1.3 Spring Boot 프로젝트 생성 및 구조

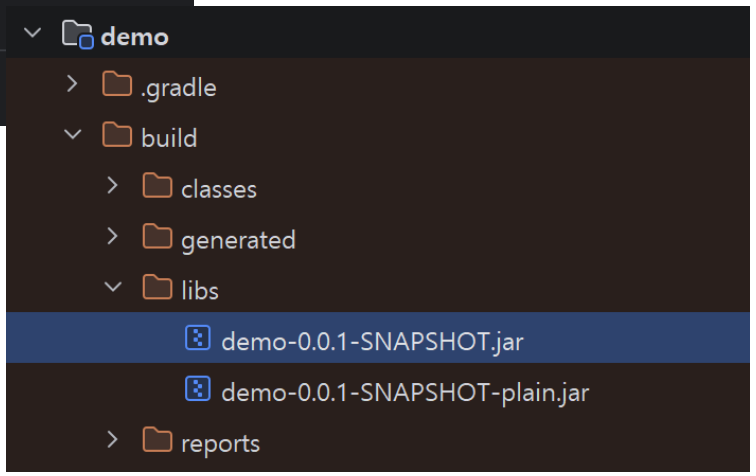
Gradle Wrapper 이용한 설치 & 실행

- 실행방법
  - A. 빌드 명령어: **./gradlew build**
  - B. 실행 명령어: **./gradlew bootRun**

```
PS C:\springboot3_study\demo> ./gradlew build
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported fo

BUILD SUCCESSFUL in 7s
7 actionable tasks: 7 executed

PS C:\springboot3_study\demo>
```



1.3 Spring Boot 프로젝트 생성 및 구조

Gradle Wrapper 이용한 설치 & 실행

```
PS C:\springboot3_study\demo> .\gradlew bootRun

> Task :bootRun


  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/

 :: Spring Boot ::                (v3.5.6)

2026-03-01T00:45:49.845+09:00  INFO 1444 --- [demo] [           main] com.example.demo.DemoAp
PID 1444 (C:\springboot3_study\demo\build\classes\java\main started by User in C:\springboot3
2026-03-01T00:45:49.847+09:00  INFO 1444 --- [demo] [           main] com.example.demo.DemoAp
 profile: "default"
```

1.3 Spring Boot 프로젝트 생성 및 구조

gradlew 실무 명령어

목적	명령어	설명
빠른 개발 실행	gradlew bootRun	소스 코드를 컴파일하고 실행. 가장 많이 사용
최초 클린 빌드	gradlew clean build	컴파일+테스트+패키징. 결과는 build/libs/*.jar 생성
테스트 생략 빌드	gradlew clean build –x test	CI에서 빠르게 산출물만 필요할 때
실행가능 jar 생성	gradlew bootJar	Boot 플러그인이 의존성 포함. 'fat JAR' 생성
특정 프로필 실행	gradlew bootRun –args="—spring.profiles.active=dev"	application-dev.yml 반영. 여러 프로필 가능. ex. dev,local,prod
테스트 상세 로그	gradlew test --info	실패 원인 추적에 용이. Junit 5 기반
의존성 트리 확인	gradlew dependencies	어떤 라이브러리가 포함되는지 확인 가능
오프라인 빌드	gradlew build --offline	네트워크 제한 환경에서 캐시만 사용
실행가능 war생성	gradlew bootWar	WAR 배포가 필요한 경우

1.3 Spring Boot 프로젝트 생성 및 구조

application.yml (.properties / .yaml)

- 개념
  - Spring Boot 실행 환경을 정의하는 외부 설정 파일
  - YAML 문법 기반의 계층형 구조
  - 실행 시 필요한 설정 값(환경변수, 서버포트, DB연결, 로깅 등)을 자동 주입

- 기능요약
  - 환경 설정
  - 실행환경 분리
  - 자동구성 제어
  - 값 주입

1.3 Spring Boot 프로젝트 생성 및 구조

YAML 문법 기초

- : 뒤에 반드시 공백 필요
- 탭(Tab)금지, 반드시 스페이스(Space)사용
- 계층구조는 들여쓰기로 표현
- 리스트는 – 하이픈으로 작성
- 문자열은 따옴표 없이 사용가능
- 주석은 # 으로 시작

1.3 Spring Boot 프로젝트 생성 및 구조

주요 설정 항목 예시

```
# 서버 및 어플리케이션 정보
server:
  port: 8080
spring:
  application:
     name: demo-app

# 데이터베이스 설정
spring:
  datasource:
        url: jdbc:mysql://localhost:3306/demo
        username: root
        password: secret
  jpa:
        hibernate:
           ddl-auto: update
        show-sql: true
```

1.3 Spring Boot 프로젝트 생성 및 구조

yml vs properties 개요

- 공통 기능
  - 두 파일 모두 Spring Boot의 환경 설정(Configuration) 파일
- 기능 역할
  - 어플리케이션 실행 시 필요한 설정값(환경변수,서버포트,DB연결,로깅 등)을 스프링 컨테이너에 자동으로 주입
- 스프링 내부 처리
  - 두 파일 모두 내부적으로 Environment 객체에 병합되어 Property Source로 관리됨

1.3 Spring Boot 프로젝트 생성 및 구조

yml vs properties 비교

항목	application.properties	application.yml
기본 형식	key=value (한 줄에 하나의 설정)	들여쓰기로 계층형 표현
가독성	key가 길어지면 가독성 떨어짐	중첩구조를 자연스럽게 표현, 가독성 향상
리스트/맵	콤마 또는 인덱스 사용 필요	- 하이픈으로 직관적으로 표현
멀티라인	₩ 이어쓰기 등 번거로움	| (파이프 기호)
프로파일 분리	application-dev.properties 등 파일분리	한 파일에서 --- 문서 분할
학습 난이도	낮음	중간 (들여쓰기 문법 주의)

1.3 Spring Boot 프로젝트 생성 및 구조

비교 예시

properties

```
server.port=8081
spring.application.name=demo-app

spring.datasource.url=jdbc:mysql://localhost:3306/demo
spring.datasource.username=root
spring.datasource.password=secret
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

yml

```
server:
  port: 8081
spring:
  application:
     name: demo-app

spring:
  datasource:
        url: jdbc:mysql://localhost:3306/demo
        username: root
        password: secret
  jpa:
        hibernate:
            ddl-auto: update
        show-sql: true
```

1.3 Spring Boot 프로젝트 생성 및 구조

DemoApplication.java

- 개념
  - Spring Boot 어플리케이션이 가장 처음 실행되는 시작점 (Entry Point) 클래스
- 역할
  - SpringApplication.run() 을 통해서 스프링 컨테이너 초기화
  - 자동 설정 (AutoConfiguration) 및 컴포넌트 스캔(Component Scan) 수행

1.4 Spring Boot 기본 어노테이션

학습목표

- @SpringBootApplication 개요
- 결합 어노테이션

  @SpringBootConfiguration
  @EnableAutoConfiguration
  @ComponentScan
- scanBasePackages 개요
- @Configuration + @Bean 개요

1.4 Spring Boot 기본 어노테이션

@SpringBootApplication

- 개념

  Spring Boot 어플리케이션의 Entry Point 클래스에 설정하는 핵심 어노테이션

  3가지 어노테이션을 결합한 메타 어노테이션

```
package com.example.demo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class DemoApplication {
   public static void main(String[] args) {
      SpringApplication.run(DemoApplication.class, args);
   }
}
```

1.4 Spring Boot 기본 어노테이션

@SpringBootApplication

- 결합 어노테이션
  - **@SpringBootConfiguration**
    스프링 설정 파일 역할 (@Configuration 포함)
  - **@EnableAutoConfiguration**
    등록된 의존성(starter)에 따라 자동 구성 수행
  - **@ComponentScan**
    현재 패키지와 하위 패키지를 스캔하여 빈(Bean) 등록

- 핵심 역할
  - 어플리케이션이 시작될 때 컴포넌트 스캔과 자동 구성을 한 번에 수행하여 최소 설정으로 실행할 수 있도록 지원

1.4 Spring Boot 기본 어노테이션

@SpringBootConfiguration

- 개념
  - @Configuration 을 포함한 스프링 설정 클래스
  - 어플리케이션 실행 시 스프링 컨테이너(ApplicationContext)에 필요한 각종 Bean 정의를 등록함.

- 특징
  - Spring Boot의 Entry Point 클래스(@SpringBootApplication)를 설정 클래스로 등록
  - 컨텍스트 초기화 시 가장 먼저 로딩
  - 내부적으로 Bean 등록, 의존성 주입, 라이프사이클 관리 등을 담당

1.4 Spring Boot 기본 어노테이션

@EnableAutoConfiguration

- 개념
  - 등록된 의존성(Starter)을 기반으로 자동 설정(Autoconfiguration) 수행

- 구성 예

Starter 의존성	자동 등록 구성
spring-boot-starter-web	Tomcat, DispatcherServlet, WebMvcConfigurer
spring-boot-starter-data-jpa	DataSource, EntityManagerFactory, TransactionManager
spring-boot-starter-security	기본 보안 필터, 로그인 페이지, 로그아웃 처리
spring-boot-starter-thymeleaf	ViewResolver, TemplateEngine
spring-boot-starter-validation	@Valid, 입력값 유효성 검증을 위한 Bean Validation 구성
mybatis-spring-boot-starter	MyBatis 설정 및 매퍼 자동 등록 (JPA 대안)
spring-boot-starter-devtools	LiveReload, 개발 중 코드 변경시 서버 자동 재시작 지원

1.4 Spring Boot 기본 어노테이션

@ComponentScan

- 개념
  - 컴포넌트 클래스들을 자동으로 스캔하여 Bean 등록
    ex. @Component, @Service, @Controller, @Repository, @Configuration 등

- 기본 규칙
  - **@SpringBootApplication이 선언된 클래스의 패키지와 하위 패키지만 스캔**
  - 다른 패키지의 Bean은 스캔 되지 않음

1.4 Spring Boot 기본 어노테이션

@ComponentScan

- 권장 패키지 구조

```
com.example.demo/
    DemoApplication.java        # @SpringBootApplication, 진입점
    domain/                     # 엔티티, 도메인모델
    service/                    # 서비스 (@Service)
    controller/                 # MVC (@Controller, @RestController)
    repository/                 # 저장소 (@Repository)
    config/                     # 설정 (@Configuration)
```

1.4 Spring Boot 기본 어노테이션

scanBasePackages / scanBasePackageClasses

- 개념
  - 기본적으로 @SpringBootApplication이 선언된 클래스 패키지와 하위 패키지만 자동 스캔됨
  - 이때 다른 패키지나 모듈에 있는 Bean 스캔이 필요한 경우 사용됨

- 문법
  - 패키지 단위

    ex. @SpringBootApplication (**scanBasePackages**={
    "com.exam", "com.shared.common", "org.external.lib" })

  - 클래스 단위

    ex. @SpringBootApplication(**scanBasePackageClasses**={
    com.exam.HelloController.class, com.shared.common.CommonConfig.class })

1.4 Spring Boot 기본 어노테이션

exclude

- 개념
  - 기본적으로 Starter 의존성에 따라 자동 구성이 수행
  - 이때 특정 기능을 직접 설정하거나 불필요한 자동 구성을 제외하고 싶을 때 사용

- 문법
  - 코드 이용

    ex. @SpringBootApplication (**exclude**={
    org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration.class })

  - YAML 이용

    ex. **spring**:
    autoconfigure:
    exclude:
    - org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration

1.4 Spring Boot 기본 어노테이션

실습 프로젝트 구조 (기본)

```
src/main/java/
    com/
        example/
            demo/
                DemoApplication.java                    # @SpringBootApplication
                config/
                    DemoConfig.java                     # @Configuration
                controller/
                    DemoController.java                 # @Controller
                repository/
                    DemoRepository.java                 # @Repository
                service/
                    DemoService.java                    # @Service
```

1.4 Spring Boot 기본 어노테이션

실습 프로젝트 구조 (scanBasePackages)

```
src/main/java/
    com/
        example/
            demo/
                DemoApplication.java                    # @SpringBootApplication
                  config/
                  controller/
                repository/
                  service/
        exam
            HelloController.java                        # @Controller
```

1.4 Spring Boot 기본 어노테이션

@Configuration

- 개념
  - 지정된 클래스는 설정(Configuration)클래스임을 알려주는 어노테이션
  - 즉, 스프링 컨테이너가 빈(Bean)으로 등록하고 관리할 설정 정보를 포함함

- 등장 배경

버전	방식	특징
Spring 2.X 이전	XML 기반 설정 ex. applicationContext.xml	설정이 복잡하고 오타 검출 어려움
Spring 3.X 이후	자바 기반 설정 (@Configuration + @Bean)	IDE 지원, 타입 안정성, 가독성 향상
Spring Boot 이후	자동 구성 + 명시적 Bean 정의 병행	최소 설정으로 빠른 개발 가능

1.4 Spring Boot 기본 어노테이션

@Bean

- 개념
  - 메서드의 반환 객체를 스프링 컨테이너의 빈(Bean)으로 등록하도록 지시하는 어노테이션
- 목적
  - XML 설정(applicationContext.xml)을 대체하여 순수 자바 코드 기반으로 Bean을 정의
- 사용 위치
  - **일반적으로 @Configuration 클래스 내부 메서드에서 사용**
- 용도
  - **자동 스캔이 불가능한 외부 라이브러리 객체, 공용 유틸, 명시적 설정이 필요한 객체 등**

1.4 Spring Boot 기본 어노테이션

@Bean 주요 특징

구분	설명
명시적 등록	자동 스캔이 불가능한 외부 객체를 직접 등록 가능
재사용성	여러 클래스에서 공통적으로 사용하는 Bean 등록 ex. RestClient, Object Mapper, PasswordEncoder, Validator 등
의존성 주입	@Autowired 또는 생성자 주입으로 쉽게 참조 가능
테스트 용이성	Mock 객체로 교체하거나 별도 테스트용 Bean 구성 가능
코드 기반 설정	XML 보다 타입 안정성과 IDE 지원이 뛰어남

1.4 Spring Boot 기본 어노테이션

Bean Scope

- 개념
  스프링 컨테이너가 Bean을 생성하고 유지하는 범위(Scope)를 의미
  @Scope 이용하여 제어 가능

scope	설명	적용 예
singleton (기본)	컨테이너 내 1개의 인스턴스만 존재	대부분의 서비스, DAO 클래스
prototype	요청할 때마다 새로운 객체 생성	매번 다른 객체를 가져야하는 객체
request	웹 요청마다 1개 인스턴스 생성	웹 컨트롤러에서 사용자별 데이터관리
session	HTTP 세션마다 1개 인스턴스 생성	로그인 정보, 사용자 세션 캐시 등

1.5 Auto Configuration과 Starter

학습목표

- Auto Configuration 개요
- 핵심 구성 요소
- 핵심 어노테이션
- Starter 개요 및 장점
- Starter 이전 비교
- 주요 Starter 종류

1.5 Auto Configuration과 Starter

Auto Configuration

- 개념
  - Spring Boot의 핵심 매커니즘으로 개발자가 직접 설정하지 않아도 필요한 환경을 자동으로 구성해주는 기능
  - 클래스패스(Classpath), 환경(Environment), 빈(Bean) 존재 여부를 조건으로 평가하여 실행시점에 적절한 설정이 자동으로 적용됨

- 특징
  - 개발 생산성 증가
  - 의존성 기반 자동 설정
  - 일관되고 표준화된 설정 구성 제공

1.5 Auto Configuration과 Starter

핵심 구성 요소

구성 요소	설명
spring-boot-autoconfigure 모듈	자동 구성 클래스 (@AutoConfiguration) 모음
@AutoConfiguration	자동 구성 클래스 지정 어노테이션 (Spring Boot 3.X 도입)
조건부 어노테이션 (@ConditionalOn* 형식)	환경 조건에 따라 Bean 등록 여부 결정 ex. @ConditionalOnClass, @ConditionalOnBean 등
AutoConfiguration.imports 파일	자동 구성 클래스를 나열해 Boot가 로드 하도록 설정
AutoConfigurationImportSelector	자동 구성 클래스 선택 및 로딩 (Spring Boot 3.X 도입)

1.5 Auto Configuration과 Starter

Starter

- 개념
  - 특정 기능(ex. Web, JPA, Security 등)을 손쉽게 사용할 수 있도록 지원하는 의존성 패키지 모음

- 특징

구분	설명
단순화	복잡한 라이브러리 의존 관계를 하나의 Starter로 해결
자동화	Auto Configuration과 연계되어 설정 코드 최소화
일관성	라이브러리 버전 일괄 관리. (명시적 버전 지정 불필요)
생산성 향상	starter 선택만으로 기능 활성화. 빠른 개발 가능
모듈화	필요한 기능만 선택 가능 (불필요한 라이브러리 로드 방지)
관례적 이름	spring-boot-starter-<기능명> ex. spring-boot-starter-web

1.5 Auto Configuration과 Starter

주요 Starter 종류

Starter 의존성	설명
spring-boot-starter	기본 코어 Starter. 로깅, 자동설정, YAML 지원 등 최소 구성 포함
spring-boot-starter-web	Spring MVC + 내장 Tomcat + REST 지원. 웹 앱 개발용
spring-boot-starter-data-jpa	JPA 기반 ORM(DB 매핑) 지원, Hibernate 포함
spring-boot-starter-security	인증/인가 기능 제공
spring-boot-starter-actuator	모니터링 및 운영 상태 점검 기능
spring-boot-starter-validation	Bean Validation(입력 검증) 기능 제공
mybatis-spring-boot-starter	MyBatis 설정 및 매퍼 자동 등록 (JPA 대안)
spring-boot-starter-devtools	LiveReload, 개발 중 코드 변경시 서버 자동 재시작 지원

1.5 Auto Configuration과 Starter

Starter 이전 비교

항목	과거 (Spring Boot 이전)	현재 (Spring Boot 이후)
설정 방법	xml 수백 라인을 직접 설정	Starter 하나로 자동 설정
버전 관리	의존성 버전 직접 관리	자동 버전 관리
Bean 등록	DispatcherServlet, Bean 직접 등록	AutoConfiguration으로 자동 Bean 주입
팀별 설정	팀별 설정 제각각	Starter로 통일된 개발 표준 지원

1.5 Auto Configuration과 Starter

대표 Starter 포함 예

```
+--- org.springframework.boot:spring-boot-starter-web -> 3.5.6
|    +--- org.springframework.boot:spring-boot-starter:3.5.6 (*)
|    +--- org.springframework.boot:spring-boot-starter-json:3.5.6
|    |    +--- org.springframework.boot:spring-boot-starter:3.5.6 (*)
|    |    +--- org.springframework:spring-web:6.2.11
|    |    |    +--- org.springframework:spring-beans:6.2.11 (*)
|    |    |    +--- org.springframework:spring-core:6.2.11 (*)
|    |    |    \--- io.micrometer:micrometer-observation:1.14.11 -> 1.15.4 (*)
|    |    +--- com.fasterxml.jackson.core:jackson-databind:2.19.2
|    |    |    +--- com.fasterxml.jackson.core:jackson-annotations:2.19.2
|    |    |    |    \--- com.fasterxml.jackson:jackson-bom:2.19.2
|    |    |    |         +--- com.fasterxml.jackson.core:jackson-annotations:2.19.2 (c)
|    |    |    |         +--- com.fasterxml.jackson.core:jackson-core:2.19.2 (c)
|    |    |    |         +--- com.fasterxml.jackson.core:jackson-databind:2.19.2 (c)
|    |    |    |         +--- com.fasterxml.jackson.datatype:jackson-datatype-jdk8:2.19.2 (c)
|    |    |    |         +--- com.fasterxml.jackson.datatype:jackson-datatype-jsr310:2.19.2 (c)
|    |    |    |         \--- com.fasterxml.jackson.module:jackson-module-parameter-names:2.19.2 (c)
|    |    |    +--- com.fasterxml.jackson.core:jackson-core:2.19.2
|    |    |    |    \--- com.fasterxml.jackson:jackson-bom:2.19.2 (*)
|    |    |    \--- com.fasterxml.jackson:jackson-bom:2.19.2 (*)
|    |    +--- com.fasterxml.jackson.datatype:jackson-datatype-jdk8:2.19.2
|    |    |    +--- com.fasterxml.jackson.core:jackson-core:2.19.2 (*)
|    |    |    +--- com.fasterxml.jackson.core:jackson-databind:2.19.2 (*)
|    |    |    \--- com.fasterxml.jackson:jackson-bom:2.19.2 (*)
|    |    +--- com.fasterxml.jackson.datatype:jackson-datatype-jsr310:2.19.2
|    |    |    +--- com.fasterxml.jackson.core:jackson-annotations:2.19.2 (*)
|    |    |    +--- com.fasterxml.jackson.core:jackson-core:2.19.2 (*)
|    |    |    +--- com.fasterxml.jackson.core:jackson-databind:2.19.2 (*)
|    |    |    \--- com.fasterxml.jackson:jackson-bom:2.19.2 (*)
|    |    \--- com.fasterxml.jackson.module:jackson-module-parameter-names:2.19.2
|    |         +--- com.fasterxml.jackson.core:jackson-core:2.19.2 (*)
|    |         +--- com.fasterxml.jackson.core:jackson-databind:2.19.2 (*)
|    |         \--- com.fasterxml.jackson:jackson-bom:2.19.2 (*)
|    +--- org.springframework.boot:spring-boot-starter-tomcat:3.5.6
|    |    +--- jakarta.annotation:jakarta.annotation-api:2.1.1
|    |    +--- org.apache.tomcat.embed:tomcat-embed-core:10.1.46
|    |    +--- org.apache.tomcat.embed:tomcat-embed-el:10.1.46
|    |    \--- org.apache.tomcat.embed:tomcat-embed-websocket:10.1.46
|    |         \--- org.apache.tomcat.embed:tomcat-embed-core:10.1.46
|    +--- org.springframework:spring-web:6.2.11 (*)
|    \--- org.springframework:spring-webmvc:6.2.11
|         +--- org.springframework:spring-aop:6.2.11 (*)
|         +--- org.springframework:spring-beans:6.2.11 (*)
|         +--- org.springframework:spring-context:6.2.11 (*)
|         +--- org.springframework:spring-core:6.2.11 (*)
|         +--- org.springframework:spring-expression:6.2.11 (*)
|         \--- org.springframework:spring-web:6.2.11 (*)
```

수고하셨습니다.